

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr. G. A. Senthil

QUESTIONS: 10

Total Marks: 50

Annexure A

RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

EXPERIMENT-1

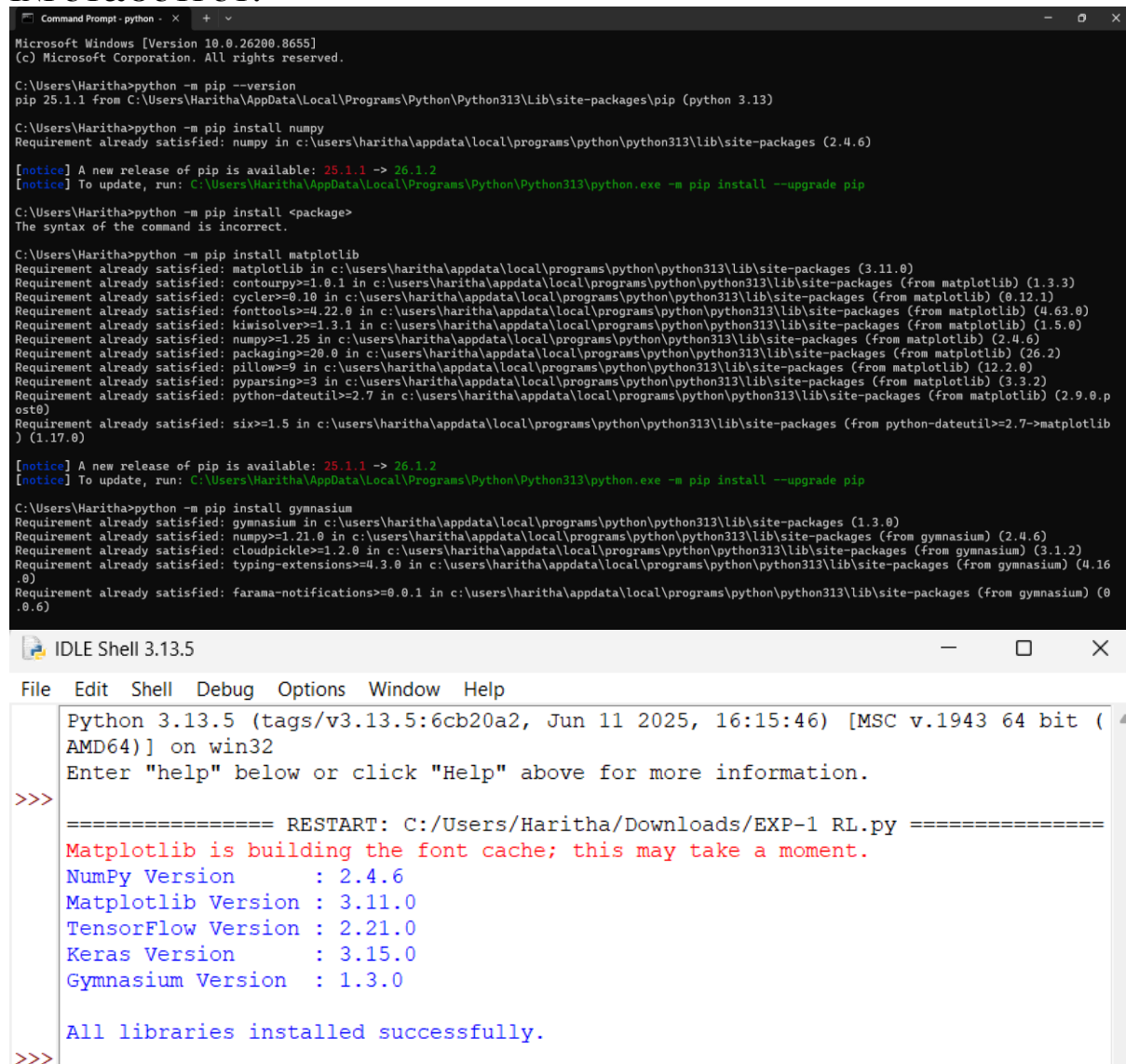
AIM:

To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

PROGRAM:

```
import numpy
import matplotlib
import tensorflow
import keras
import gymnasium
print("NumPy Version :", numpy.__version__)
print("Matplotlib Version :", matplotlib.__version__)
print("TensorFlow Version :", tensorflow.__version__)
print("Keras Version :", keras.__version__)
print("Gymnasium Version :", gymnasium.__version__)
print("\nAll libraries installed successfully.")
```

INPUT&OUTPUT:



```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Haritha>python -m pip --version
pip 25.1.1 from C:\Users\Haritha\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)

C:\Users\Haritha>python -m pip install numpy
Requirement already satisfied: numpy in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (2.4.6)

[notice] A new release of pip is available: 25.1.1 -> 26.1.2
[notice] To update, run: C:\Users\Haritha\AppData\Local\Programs\Python\Python313\python.exe -m pip install --upgrade pip

C:\Users\Haritha>python -m pip install <package>
The syntax of the command is incorrect.

C:\Users\Haritha>python -m pip install matplotlib
Requirement already satisfied: matplotlib in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (3.11.0)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (2.4.6)
Requirement already satisfied: packaging>=20.0 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

[notice] A new release of pip is available: 25.1.1 -> 26.1.2
[notice] To update, run: C:\Users\Haritha\AppData\Local\Programs\Python\Python313\python.exe -m pip install --upgrade pip

C:\Users\Haritha>python -m pip install gymnasium
Requirement already satisfied: gymnasium in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (1.3.0)
Requirement already satisfied: numpy>=1.21.0 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from gymnasium) (2.4.6)
Requirement already satisfied: cloudpickle>=1.2.0 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from gymnasium) (3.1.2)
Requirement already satisfied: typing-extensions>=4.3.0 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from gymnasium) (4.16.0)
Requirement already satisfied: farama-notifications>=0.0.1 in c:\users\haritha\appdata\local\programs\python\python313\lib\site-packages (from gymnasium) (0.0.6)

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Haritha/Downloads/EXP-1 RL.py =====
Matplotlib is building the font cache; this may take a moment.
NumPy Version      : 2.4.6
Matplotlib Version : 3.11.0
TensorFlow Version : 2.21.0
Keras Version      : 3.15.0
Gymnasium Version  : 1.3.0

All libraries installed successfully.
>>>
```

EXPERIMENT -2

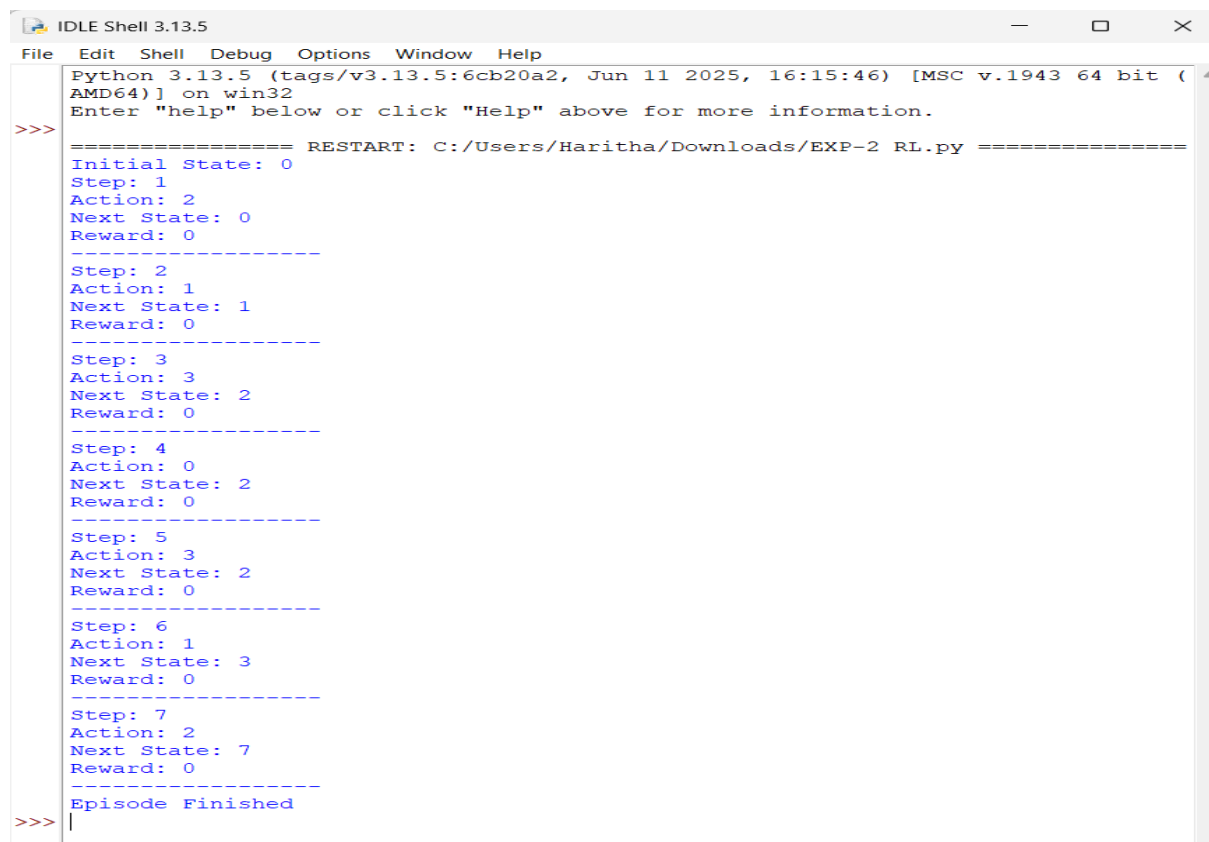
AIM:

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

PROGRAM:

```
import gymnasium as gym
# Create the FrozenLake environment
env = gym.make("FrozenLake-v1")
state, info = env.reset()
print("Initial State:", state)
for step in range(10):
    print("Next State:", next_state)
    print("Reward:", reward)
    print("-----") state = next_state
    if terminated or truncated:
        print("Episode Finished")
        break
env.close()
```

INPUT&OUTPUT:



```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Haritha/Downloads/EXP-2 RL.py =====
Initial State: 0
Step: 1
Action: 2
Next State: 0
Reward: 0
-----
Step: 2
Action: 1
Next State: 1
Reward: 0
-----
Step: 3
Action: 3
Next State: 2
Reward: 0
-----
Step: 4
Action: 0
Next State: 2
Reward: 0
-----
Step: 5
Action: 3
Next State: 2
Reward: 0
-----
Step: 6
Action: 1
Next State: 3
Reward: 0
-----
Step: 7
Action: 2
Next State: 7
Reward: 0
-----
Episode Finished
>>> |
```

EXPERIMENT-3

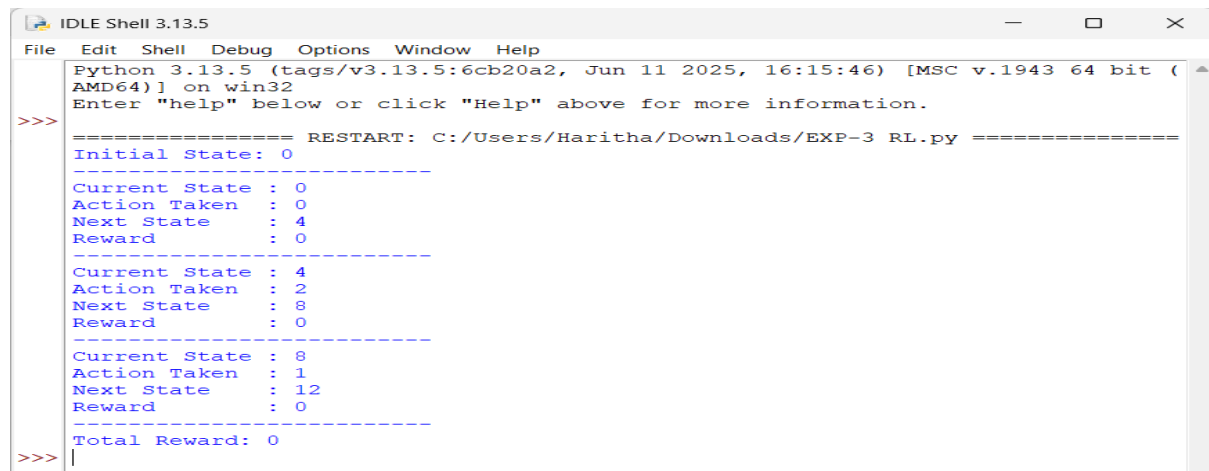
AIM:

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

PROGRAM:

```
import gymnasium as gym
env = gym.make("FrozenLake-v1")
state, info = env.reset()
print("Initial State:", state)
total_reward = 0
done = False
while not done:
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)
    print("-----")
    print("Current State :", state)
    print("Action Taken :", action)
    print("Next State  :", next_state)
    print("Reward      :", reward)
    state = next_state
    total_reward += reward
    done = terminated or truncated
print("-----")
print("Total Reward:", total_reward)
env.close()
```

OUTPUT:



```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Haritha/Downloads/EXP-3 RL.py =====
Initial State: 0
-----
Current State : 0
Action Taken  : 0
Next State    : 4
Reward        : 0
-----
Current State : 4
Action Taken  : 2
Next State    : 8
Reward        : 0
-----
Current State : 8
Action Taken  : 1
Next State    : 12
Reward        : 0
-----
Total Reward: 0
>>> |
```

EXPERIMENT-4

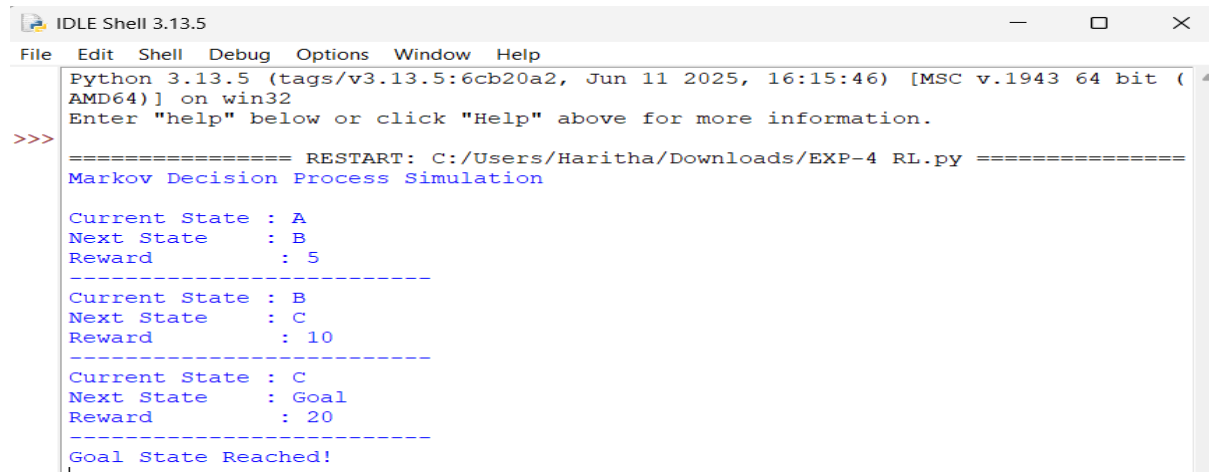
AIM:

To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

PROGRAM:

```
states = ["A", "B", "C", "Goal"]
# Policy (State Transitions)
policy = {
    "A": "B",
    "B": "C",
    "C": "Goal"
}
rewards = {
    ("A", "B"): 5,
    ("B", "C"): 10,
    ("C", "Goal"): 20
}
current_state = "A"
print("Markov Decision Process Simulation\n")
while current_state != "Goal":
    next_state = policy[current_state]
    reward = rewards[(current_state, next_state)]
    print("Current State :", current_state)
    print("Next State   :", next_state)
    print("Reward       :", reward)
    print("-----")
    current_state = next_state
print("Goal State Reached!")
```

OUTPUT:



```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Haritha/Downloads/EXP-4 RL.py =====
Markov Decision Process Simulation

Current State : A
Next State : B
Reward : 5
-----
Current State : B
Next State : C
Reward : 10
-----
Current State : C
Next State : Goal
Reward : 20
-----
Goal State Reached!
~~~~
```

EXPERIMENT-5

AIM:

To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

PROGRAM:

```
# Discount Factor
gamma = 0.9

# Immediate Reward
reward = 10

# Next State Value
next_state_value = 20

# Bellman Expectation Equation
state_value = reward + gamma * next_state_value
print("Bellman Expectation Equation")
print("-----")
print("Reward =", reward)
print("Discount Factor =", gamma)
print("Next State Value =", next_state_value)
print("Current State Value =", state_value)
print("\nBellman Optimality Equation")
print("-----")

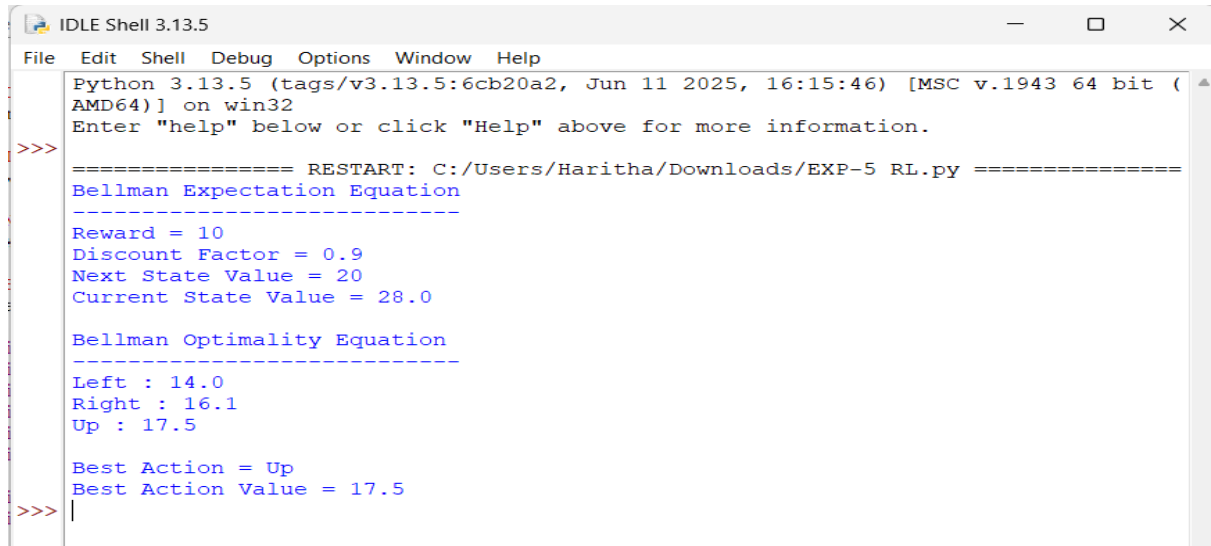
# Action Values
actions = {
    "Left": 5 + gamma * 10,
    "Right": 8 + gamma * 9,
    "Up": 4 + gamma * 15
```

```

}
for action, value in actions.items():
    print(action, ":", value)
best_action = max(actions, key=actions.get)
print("\nBest Action =", best_action)
print("Best Action Value =", actions[best_action])

```

OUTPUT:



```

IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Haritha/Downloads/EXP-5 RL.py =====
Bellman Expectation Equation
-----
Reward = 10
Discount Factor = 0.9
Next State Value = 20
Current State Value = 28.0

Bellman Optimality Equation
-----
Left : 14.0
Right : 16.1
Up : 17.5

Best Action = Up
Best Action Value = 17.5
>>>

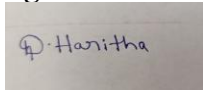
```

Code of Conduct:

I **Dasari Haritha(192425254)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand